

屏幕显示（SPI）

屏幕显示（SPI）

一、学习目标

SPI协议

SPI硬件接口

SPI优点

SPI确定

二、硬件搭建

三、实验步骤

1. 打开SYSCONFIG配置工具

2. SPI参数配置

3. 引脚参数配置

LCD屏幕

外部Flash

3. SPI协议使用

4. 写入程序

5. 编译

四、程序分析

五、实验现象

一、学习目标

1. 学习了解spi通讯基本知识。

2. 读取flash内数据并在板载LCD屏幕上显示。

SPI协议

SPI协议（串行外设接口）是一种常用的同步串行通信协议，专为微控制器与外部设备之间的高速、短距离通信而设计。它支持全双工通信，即数据可以在主设备和从设备之间同时发送和接收，常用于存储器芯片、传感器、显示驱动器和无线模块等设备。

SPI硬件接口

SPI协议通常使用四条线进行数据传输：

- **SCLK**（串行时钟）：主设备生成的时钟信号，用于同步通信。
- **MOSI**（主输出从输入）：主设备发送数据给从设备。
- **MISO**（主输入从输出）：从设备发送数据给主设备。
- **SS/CS**（从设备选择）：主设备通过片选信号激活目标从设备。

SPI优点

- 通信速度快，适合高带宽应用。
- 支持全双工传输，提高效率。
- 硬件实现简单，接口直接。
- 支持多个从设备连接。

SPI确定

- 缺乏标准化的错误检测机制。
- 主设备引脚数随从设备数量增加。
- 通信距离受限制，通常用于板载通信。

二、硬件搭建

W25Q32是一款常见的串行闪存器件，采用SPI（Serial Peripheral Interface）接口协议。它具有高速的读写和擦除功能，广泛应用于嵌入式系统、存储设备、路由器等高性能电子设备中。W25Q32的容量为32 Mbit（即4 MB），型号中的数字代表了不同的容量选项，例如W25Q16、W25Q64、W25Q128等，以满足不同应用场景的需求。

W25Q32芯片的内存是按扇区（Sector）和块（Block）进行分配的。具体来说：

- 扇区（**Sector**）：每个扇区的大小为4KB。
- 块（**Block**）：每个块包含16个扇区，即一个块的大小为64KB。

这些存储单元结构使得数据管理和存储操作更加灵活，特别适合需要频繁读写操作的嵌入式系统和存储应用。

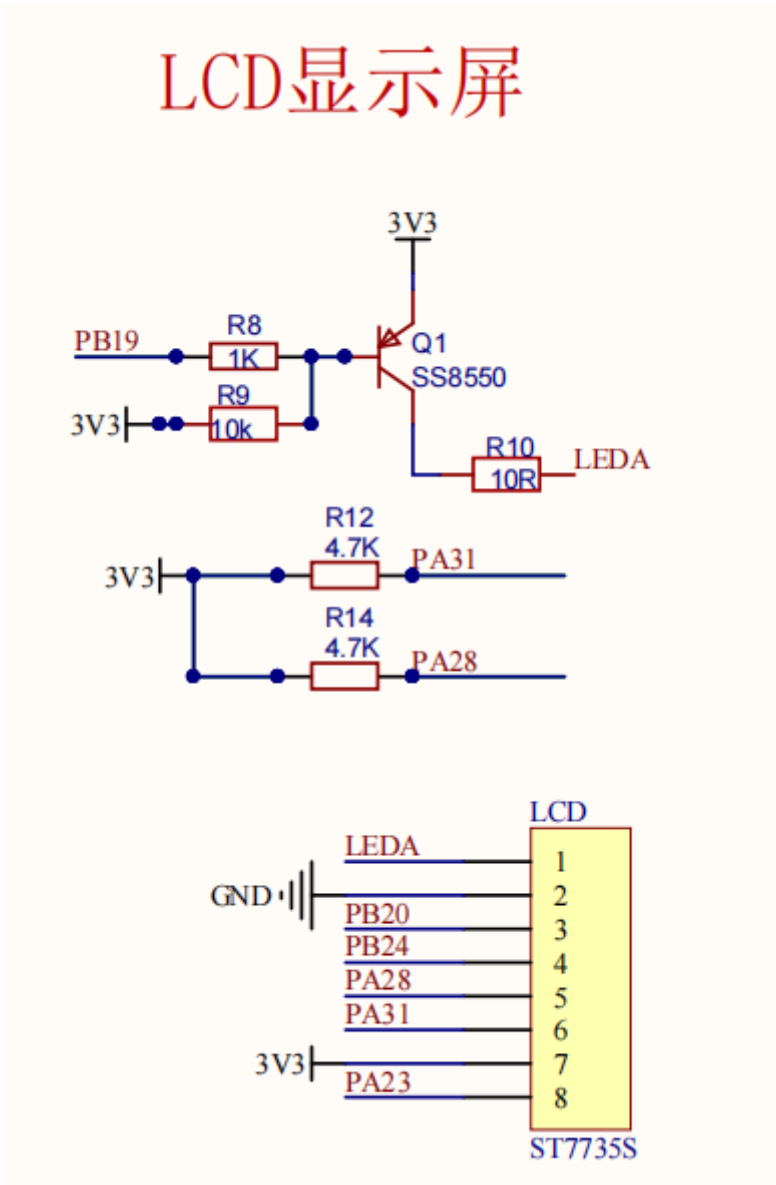
我们使用的是硬件SPI方式驱动W25Q32，因此我们需要确定我们设置的引脚是否有硬件SPI外设接口。在数据手册中，PB14~PB17可以复用为SPI1的4根通信线。

31	PB14		SPI1_CS3 [2] / SPI1_POCI [3] / SPI0_CS3 [4] / TIMG12_C1 [5] / TIMG8_IDX [6] / TIMA0_C0 [7]	2	24	-	-	标准
32	PB15		UART2_TX [2] / SPI1_PICO [3] / UART3_CTS [4] / TIMG8_C0 [5] / TIMG7_C0 [6]	3	25	-	-	标准
33	PB16		UART2_RX [2] / SPI1_SCK [3] / UART3_RTS [4] / TIMG8_C1 [5] / TIMG7_C1 [6]	4	26	-	-	标准
43	PB17	A1_4 / COMP1_IN2-	UART2_TX [2] / SPI0_PICO [3] / SPI1_CS1 [4] / TIMA1_C0 [5] / TIMA0_C2 [6]	14	36	-	-	标准

本次课程无需额外的硬件设备，直接利用MSPM0G3507主板上的板载LCD屏幕和外部存储flash即可。

MSPM0G3507主控图：

LCD显示屏



三、实验步骤

这里使用我们提供的模板工程进行介绍。

将模板工程放在SDK的根目录下解压，我将其重新命名为11_SPI文件夹。

新加卷 (D:) > ti > mspm0_sdk_2_02_00_05					在 n
	名称	修改日期	类型	大小	
C)	01_LED	2024/11/19 11:42	文件夹		
	02_Delay	2024/11/19 18:29	文件夹		
	03_KEY	2024/11/19 20:09	文件夹		
	04_ExtInterrupt	2024/11/19 21:15	文件夹		
	05_UART	2024/11/20 19:41	文件夹		
	06_Timer	2024/11/20 20:28	文件夹		
	07_PWM	2024/11/21 15:59	文件夹		
	08_ADC	2024/11/21 18:37	文件夹		
	09_DMA	2024/11/21 19:48	文件夹		
	10_I2C	2024/11/26 14:08	文件夹		
	11_SPI	2024/11/26 14:20	文件夹		
	docs	2024/11/14 15:05	文件夹		
	examples	2024/11/14 15:05	文件夹		
	kernel	2024/11/14 15:09	文件夹		

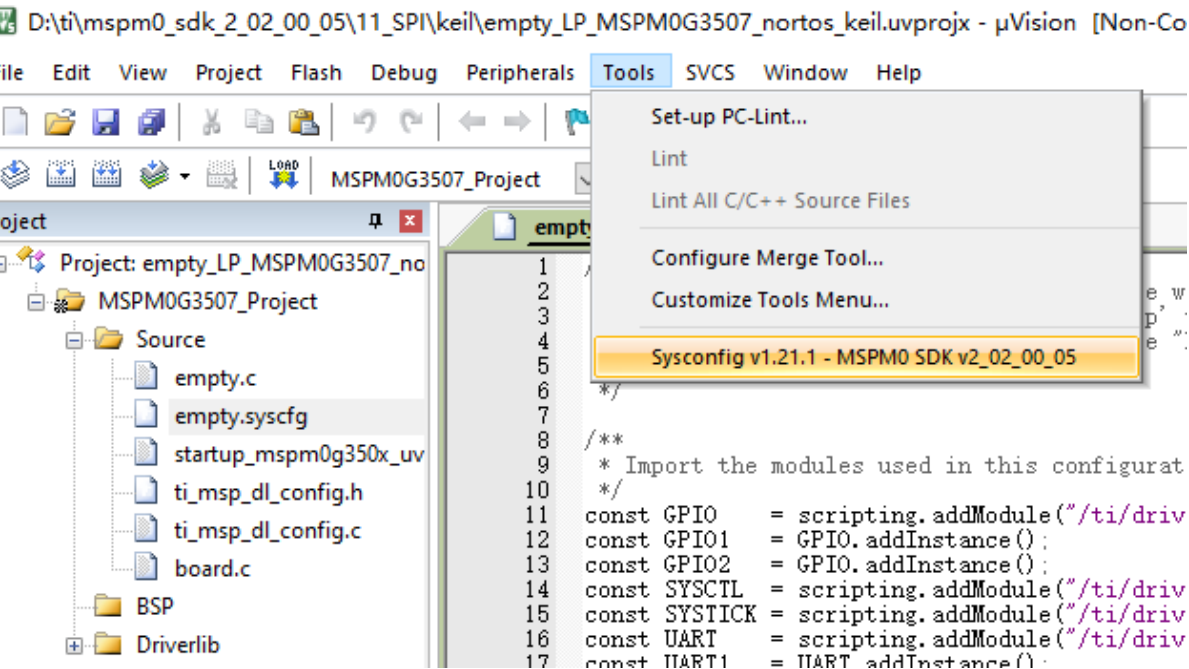
1. 打开SYSCONFIG配置工具

接着打开该工程。

ti > mspm0_sdk_2_02_00_05 > 11_SPI > keil

名称	修改日期	类型	大小
empty LP MSPM0G3507 nortos keil....	2024/6/26 14:16	UVOPTX 文件	21 KB
empty LP MSPM0G3507 nortos keil....	2024/6/26 11:07	UVision5 Project	20 KB
mspm0g3507.sct	2024/1/25 11:42	Windows Script ...	1 KB
startup_mspm0g350x_uvision.s	2024/1/25 11:42	Assembler Source	10 KB

在Keil的主界面打开empty.syscfg文件，在empty.syscfg文件打开的情况下，再打开SYSCONFIG的GUI界面。



在左侧找到 SPI 栏，点击进入，添加SPI外设。

FILEABOUT

Type Filter Text...

← → Software > SPI

MSPM0 DRIVER LIBRARY ...

SYSTEM (9)

Board1/1✓⊕

Configuration NVM⊕

DMA⊕

GPIO7✓⊕

MATHACL⊕

RTC⊕

SYSCTL1/1✓⊕

SYSTICK1/1✓⊕

WWDT⊕

ANALOG (6)

ADC12⊕

COMP⊕

DAC12⊕

GPAMP⊕

OPA⊕

VREF⊕

COMMUNICATIONS (6)

I2C⊕

I2C - SMBUS⊕

MCAN⊕

SPI1/2✓⊕

UART1/4✓⊕

UART - LIN⊕

TIMERS (6)

SPI (1 of 2 Added) ⓘ

✓ SPI⊗

Peripheral does not retain register contents in STOP or STANDBY modes. User should take care to save and restore register configuration in application. See Retention Configuration section for more details.

NameSPI

Selected PeripheralSPI1

Quick Profiles

SPI ProfilesCustom

Basic Configuration

SPI Initialization Configuration

Mode SelectController

Clock Configuration

Target Bit Rate (Hz)16000000

Calculated Bit Rate16000000.00

Calculated Error (%)0

2. SPI参数配置

RX FIFO Threshold Level: 配置RX的FIFO中断触发等级。配置为RX FIFO contains >= 2 entries

TX FIFO Threshold Level: 配置TX的FIFO中断触发等级。配置为TX FIFO contains <= 2 entries

Communication Direction: SPI通信方向。配置为PICO and PoCI, 既发送也接收

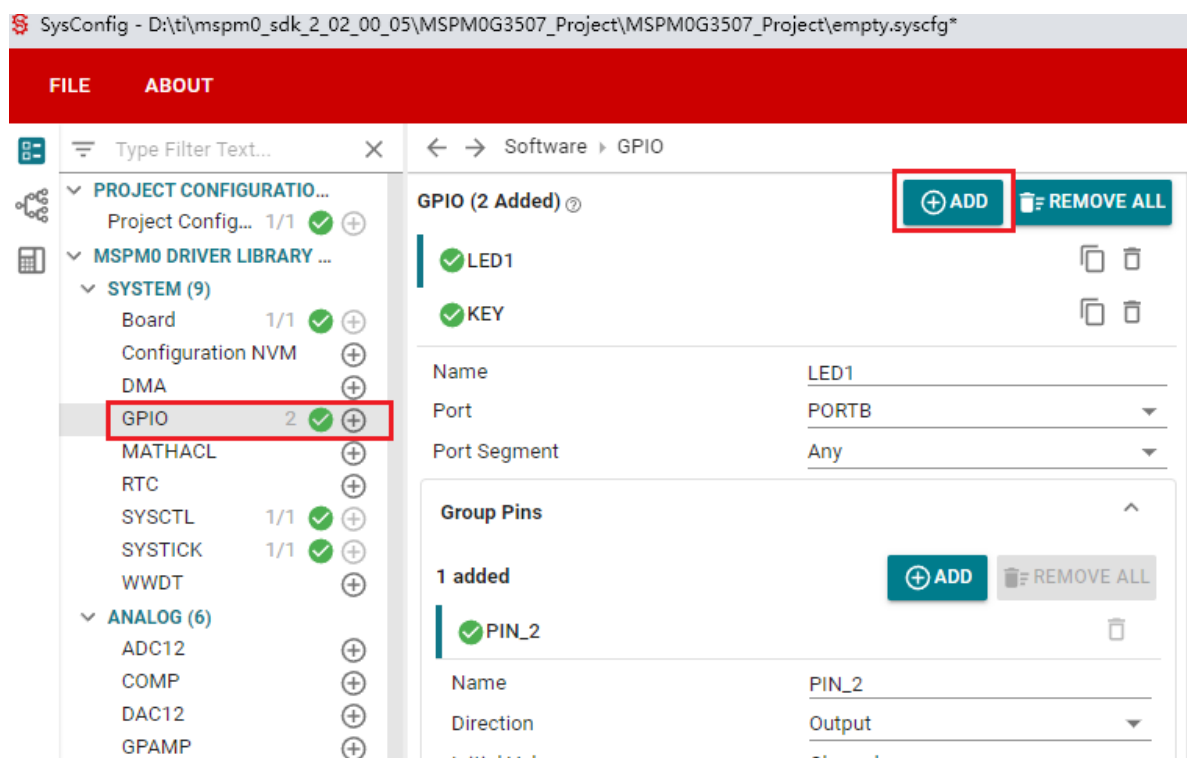
Enable Packing: 是否开启32位发送方式。这里不开启

PinMux Peripheral and Pin Configuration			
SPI Peripheral	SPI1		
SPI SCLK (Clock)	PB16/4		
SPI PICO (Peripheral In, Controller Out)	PB15/3		
SPI POCl (Peripheral Out, Controller In)	PB14/2		
SPI CS1 (Chip Select 1)	Any(PA27/31)		

CS引脚先随便配置，后面我们是使用软件方式的CS。

3. 引脚参数配置

在左侧找到 GPIO 栏，点击进入，添加几组GPIO。



LCD屏幕

背光 **BLK:**

GPIO (6 Added) ?

+ ADD

REMOVE ALL

✓ BLK

✓ CS

✓ DC

✓ RES

✓ MOSI

✓ SCLK

Name

BLK

Port

PORTB

Port Segment

Any

Group Pins

^

1 added

+ ADD

REMOVE ALL

✓ PIN_19

Name

PIN_19

Direction

Output

Initial Value

Set

IO Structure

Any

Digital IOMUX Features

▼

Assigned Port

PORTB

Assigned Port Segment

Any

Assigned Pin

19

Interrupts/Events

▼

CS、DC、RES、MOSI、SCLK分别如下：

GPIO (6 Added) ?

+ ADD

REMOVE ALL

✓ BLK

✓ CS

✓ DC

✓ RES

✓ MOSI

✓ SCLK

Name

CS

Port

PORTA

Port Segment

Any

Group Pins

^

1 added

+ ADD

REMOVE ALL

✓ PIN_23

Name

PIN_23

Direction

Output

Initial Value

Cleared

IO Structure

Any

Digital IOMUX Features

▼

Assigned Port

PORTA

Assigned Port Segment

Any

Assigned Pin

23

Interrupts/Events

▼

GPIO (6 Added) ?

+ ADD

REMOVE ALL

✓ BLK

✓ CS

✓ DC

✓ RES

✓ MOSI

✓ SCLK

Name

DC

Port

PORTB

Port Segment

Any

Group Pins

^

1 added

+ ADD

REMOVE ALL

✓ PIN_24

Name

PIN_24

Direction

Output

Initial Value

Cleared

IO Structure

Any

Digital IOMUX Features

▼

Assigned Port

PORTB

Assigned Port Segment

Any

Assigned Pin

24

Interrupts/Events

▼

GPIO (6 Added) ?

+ ADD

REMOVE ALL

✓ BLK

✓ CS

✓ DC

✓ RES

✓ MOSI

✓ SCLK

Name

RES

Port

PORTB

Port Segment

Any

Group Pins

^

1 added

+ ADD

REMOVE ALL

✓ PIN_20

Name

PIN_20

Direction

Output

Initial Value

Cleared

IO Structure

Any

Digital IOMUX Features

▼

Assigned Port

PORTB

Assigned Port Segment

Any

Assigned Pin

20

Interrupts/Events

▼

GPIO (6 Added) ⓘ

+

ADD

REMOVE ALL

✓

BLK

✓

CS

✓

DC

✓

RES

✓

MOSI

✓

SCLK

✓

BLK

✓

CS

✓

DC

✓

RES

✓

MOSI

✓

SCLK

Name

SCLK

Port

PORTA

Port Segment

Any

Group Pins

^

1 added

+

ADD

REMOVE ALL

✓

PIN_31

✓

BLK

✓

CS

✓

DC

✓

RES

✓

MOSI

✓

SCLK

✓

PIN_31

✓

BLK

✓

CS

✓

DC

✓

RES

✓

MOSI

✓

SCLK

Name

PIN_31

Direction

Output

Initial Value

Cleared

IO Structure

Any

Digital IOMUX Features

▼

Assigned Port

PORTA

Assigned Port Segment

Any

Assigned Pin

31

Interrupts/Events

▼

GPIO (6 Added) ⓘ

+

ADD

REMOVE ALL

✓

BLK

✓

CS

✓

DC

✓

RES

✓

MOSI

✓

SCLK

✓

BLK

✓

CS

✓

DC

✓

RES

✓

MOSI

✓

SCLK

Name

MOSI

Port

PORTA

Port Segment

Any

Group Pins

^

1 added

+

ADD

REMOVE ALL

✓

PIN_28

✓

BLK

✓

CS

✓

DC

✓

RES

✓

MOSI

✓

SCLK

✓

PIN_28

✓

BLK

✓

CS

✓

DC

✓

RES

✓

MOSI

✓

SCLK

Name

PIN_28

Direction

Output

Initial Value

Cleared

IO Structure

Any

Digital IOMUX Features

▼

Assigned Port

PORTA

Assigned Port Segment

Any

Assigned Pin

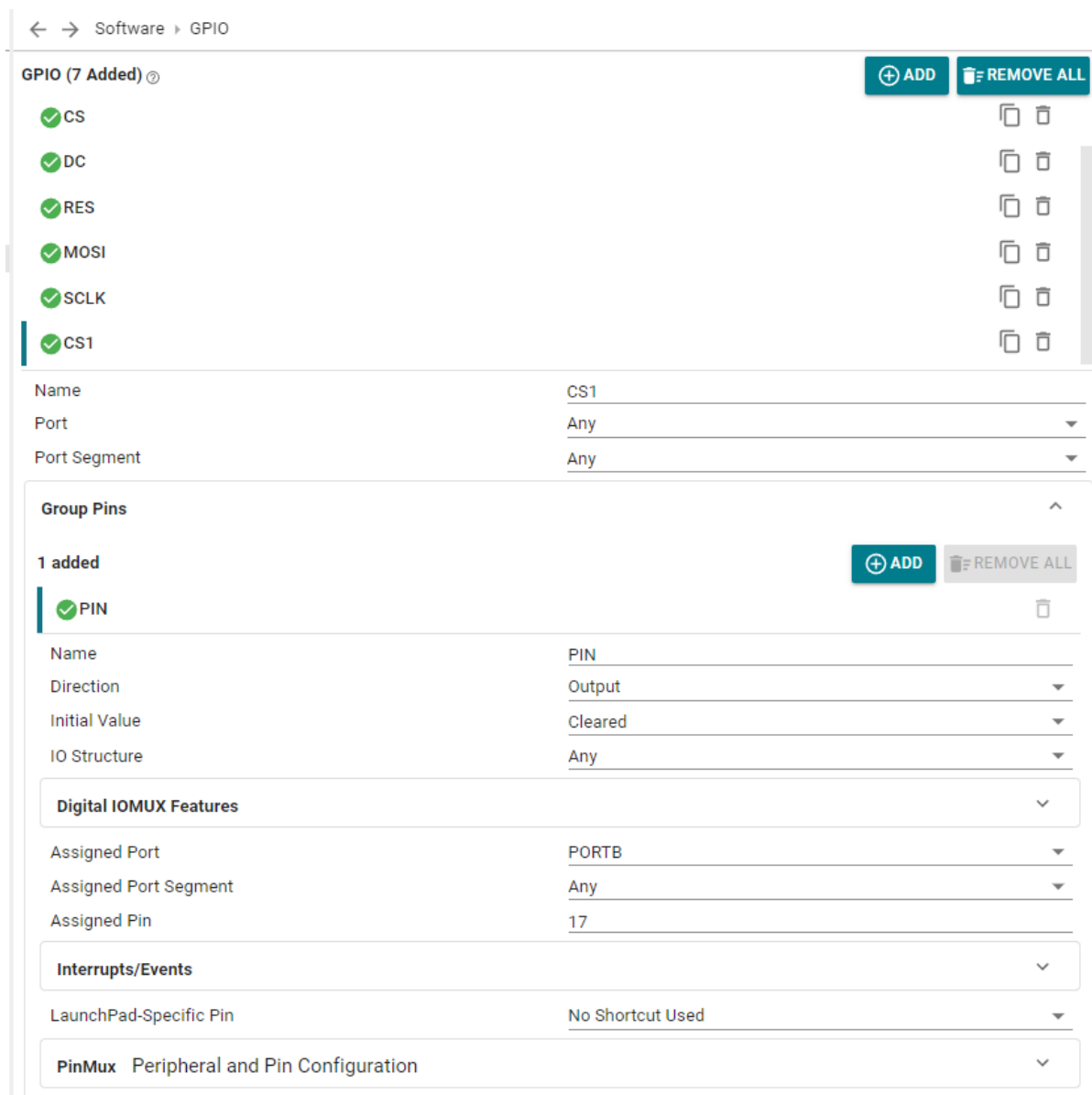
28

Interrupts/Events

▼

外部Flash

CS1



保存退出。

3. SPI协议使用

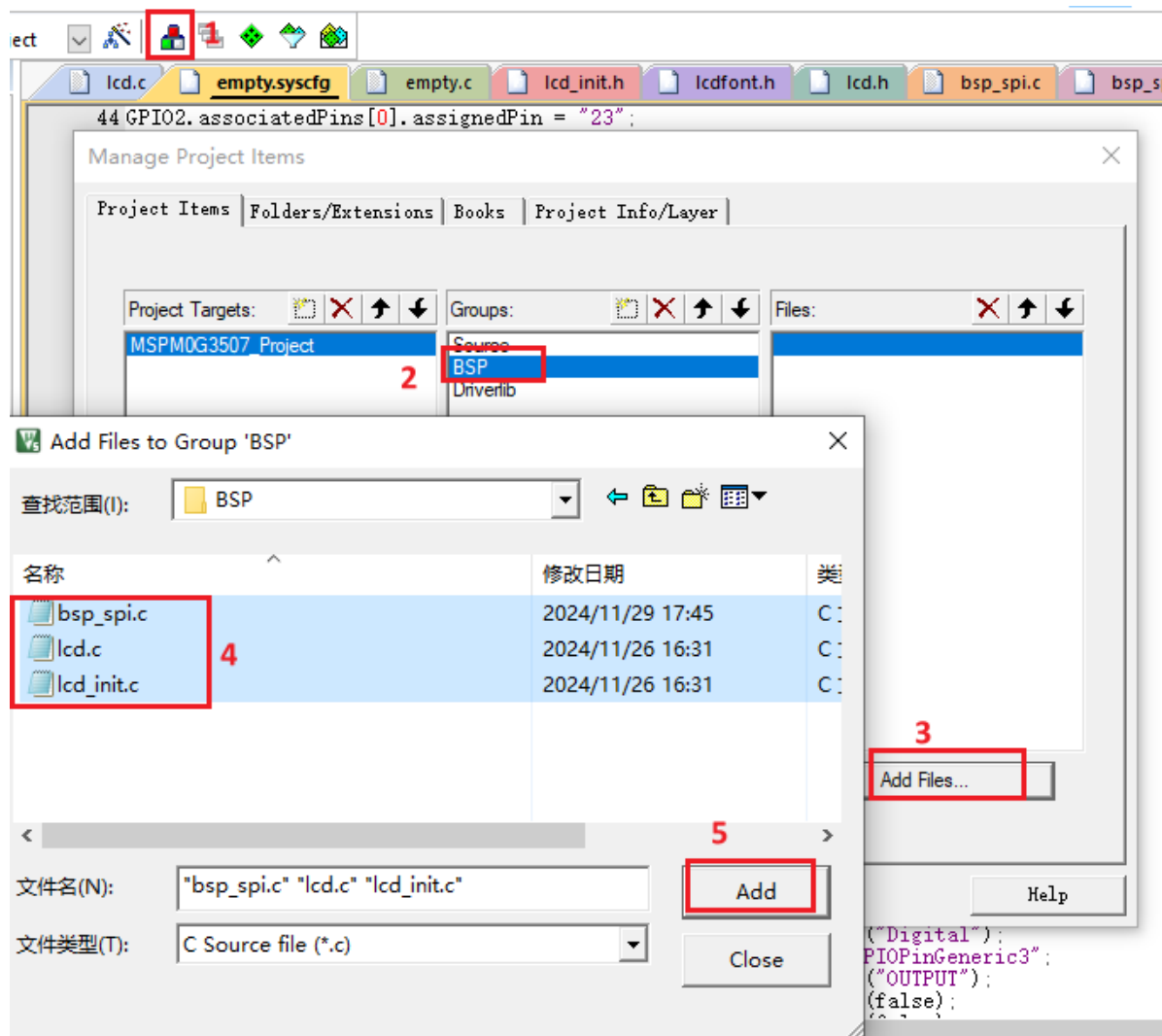
我们在工程模板中新建四个文件，分别是 lcd.c、lcd.h、bsp_spi.c 和 bsp_spi.h。将它们保存在工程的 BSP 文件夹下。

包括以下这些LCD显示屏的驱动程序和字库，放入BSP文件夹中

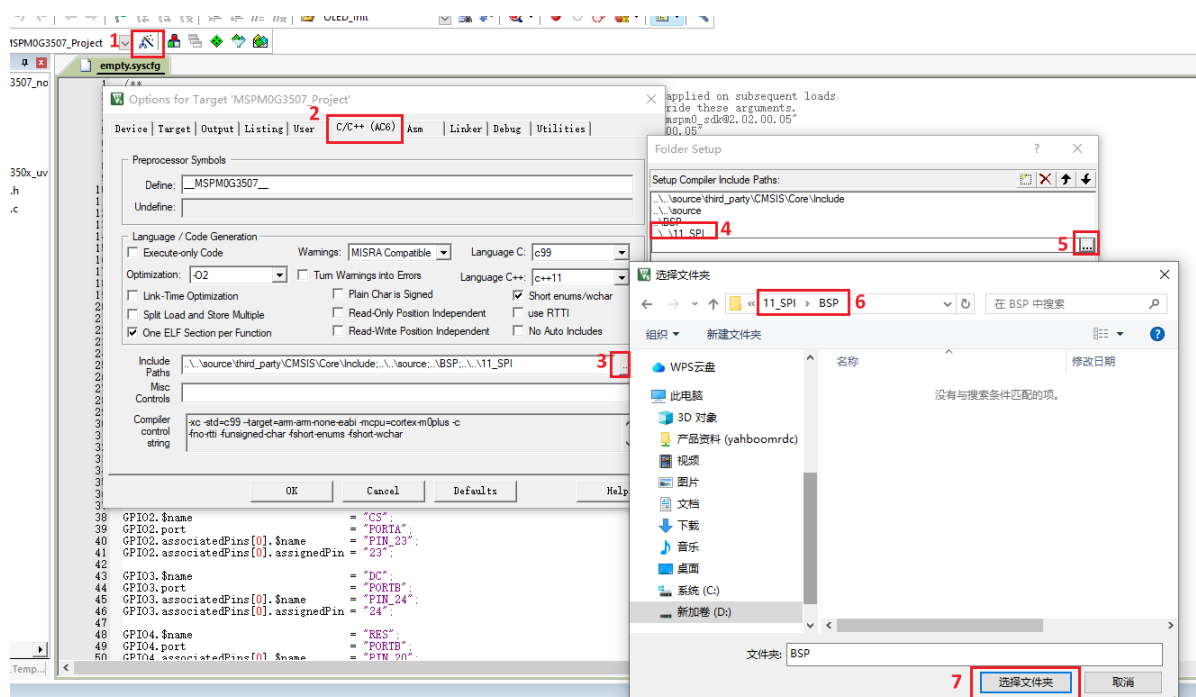
:) > ti > mspm0_sdk_2_02_00_05 > 11_SPI > BSP					在 BSP .
名称	修改日期	类型	大小		
bsp_spi.c	2024/11/29 17:45	C 文件	5 KB		
bsp_spi.h	2024/11/29 17:31	C Header 源文件	1 KB		
flash_use.c	2024/11/29 17:50	C 文件	2 KB		
lcd.c	2024/11/26 16:31	C 文件	11 KB		
lcd.h	2024/11/26 16:06	C Header 源文件	3 KB		
lcd_init.c	2024/11/26 16:31	C 文件	7 KB		
lcd_init.h	2024/11/21 16:33	C Header 源文件	3 KB		
lcdfont.h	2023/1/9 16:23	C Header 源文件	72 KB		
pic.h	2023/1/9 22:57	C Header 源文件	52 KB		

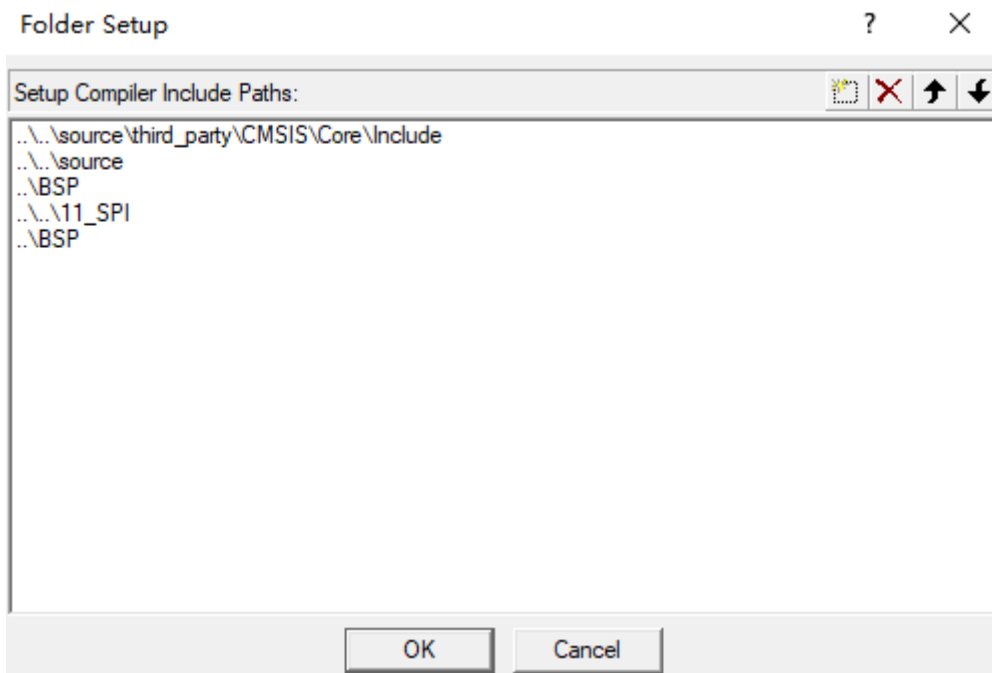
字库

打开项目管理，点击BSP，在BSP中添加我们之前新建和复制过来的文件。



更新头文件路径。





4. 写入程序

定义LCD显示屏的操作接口和相关功能

lcd.h

```
#ifndef __LCD_H
#define __LCD_H
#include <stdint.h>

void LCD_Fill(uint16_t xsta,uint16_t ysta,uint16_t xend,uint16_t yend,uint16_t
color);//指定区域填充颜色 Specify area fill color
void LCD_DrawPoint(uint16_t x,uint16_t y,uint16_t color);//在指定位置画一个点 Draw a
point at the specified location
void LCD_DrawLine(uint16_t x1,uint16_t y1,uint16_t x2,uint16_t y2,uint16_t
color);//在指定位置画一条线 Draw a line at the specified position
void LCD_DrawRectangle(uint16_t x1, uint16_t y1, uint16_t x2, uint16_t
y2,uint16_t color);//在指定位置画一个矩形 Draw a rectangle at the specified location
void Draw_Circle(uint16_t x0,uint16_t y0,uint8_t r,uint16_t color);//在指定位置画一
个圆 Draw a circle at the specified location

void LCD_ShowChinese(uint16_t x,uint16_t y,uint8_t *s,uint16_t fc,uint16_t
bc,uint8_t sizey,uint8_t mode);//显示汉字串 Display Chinese character string
void LCD_ShowChinese12x12(uint16_t x,uint16_t y,uint8_t *s,uint16_t fc,uint16_t
bc,uint8_t sizey,uint8_t mode);//显示单个12x12汉字 Display a single 12x12 Chinese
character
void LCD_ShowChinese16x16(uint16_t x,uint16_t y,uint8_t *s,uint16_t fc,uint16_t
bc,uint8_t sizey,uint8_t mode);//显示单个16x16汉字 Display a single 16x16 Chinese
character
void LCD_ShowChinese24x24(uint16_t x,uint16_t y,uint8_t *s,uint16_t fc,uint16_t
bc,uint8_t sizey,uint8_t mode);//显示单个24x24汉字 Display a single 24x24 Chinese
character
void LCD_ShowChinese32x32(uint16_t x,uint16_t y,uint8_t *s,uint16_t fc,uint16_t
bc,uint8_t sizey,uint8_t mode);//显示单个32x32汉字 Display a single 32x32 Chinese
character
```

```

void LCD_ShowChar(uint16_t x,uint16_t y,uint8_t num,uint16_t fc,uint16_t
bc,uint8_t sizey,uint8_t mode);//显示一个字符 Display a character
void LCD_ShowString(uint16_t x,uint16_t y,const uint8_t *p,uint16_t fc,uint16_t
bc,uint8_t sizey,uint8_t mode);//显示字符串 Display String
uint32_t mypow(uint8_t m,uint8_t n);//求幂 Power
void LCD_ShowIntNum(uint16_t x,uint16_t y,uint16_t num,uint8_t len,uint16_t
fc,uint16_t bc,uint8_t sizey);//显示整数变量 Display integer variables
void LCD_ShowFloatNum1(uint16_t x,uint16_t y,float num,uint8_t len,uint16_t
fc,uint16_t bc,uint8_t sizey);//显示两位小数变量 Display variables with two decimal
places

void LCD_ShowPicture(uint16_t x,uint16_t y,uint16_t length,uint16_t width,const
uint8_t pic[]);//显示图片 Show image

//画笔颜色 Brush Color
#define WHITE          0xFFFF
#define BLACK          0x0000
#define BLUE           0x001F
#define BRED           0XF81F
#define GRED           0xFFE0
#define GBLUE          0x07FF
#define RED            0xF800
#define MAGENTA        0xF81F
#define GREEN          0x07E0
#define CYAN           0x7FFF
#define YELLOW         0xFFE0
#define BROWN          0XBC40 //棕色 Brown
#define BRRED          0XFC07 //棕红色 Brown red
#define GRAY           0X8430 //灰色 Gray
#define DARKBLUE       0X01CF //深蓝色 Dark blue
#define LIGHTBLUE      0X7D7C //浅蓝色 Light blue
#define GRAYBLUE       0X5458 //灰蓝色 Gray blue
#define LIGHTGREEN     0X841F //浅绿色 Light green
#define LGRAY          0XC618 //浅灰色(PANNEL),窗体背景色 Light gray (PANNEL),
window background color
#define LGRAYBLUE      0XA651 //浅灰蓝色(中间层颜色) Light gray blue (middle layer
color)
#define LBBLUE         0X2B12 //浅棕蓝色(选择条目的反色) Light brown blue
(inverted color of selected item)

#endif

```

接下来是 LCD显示屏驱动程序

lcd.c (这里只截取部分，详细内容请查看工程源码)

```

#include "lcd.h"
#include "lcd_init.h"
#include "lcdfont.h"
#include "board.h"

/*****
函数说明：在指定区域填充颜色
入口数据：xsta,ysta 起始坐标
           xend,yend 终止坐标
*****/

```

```

        color          要填充的颜色

返回值： 无
Function description: Fill the specified area with color
Input data: xsta, ysta starting coordinates
            xend, yend ending coordinates
            color the color to be filled

Return value: None
*****/
void LCD_Fill(uint16_t xsta,uint16_t ysta,uint16_t xend,uint16_t yend,uint16_t
color)
{
    uint16_t i,j;
    LCD_Address_Set(xsta,ysta,xend-1,yend-1);//设置显示范围 Set the display range
    for(i=ysta;i<yend;i++)
    {
        for(j=xsta;j<xend;j++)
        {
            LCD_WR_DATA(color);
        }
    }
}

/*****
函数说明：在指定位置画点
入口数据：x,y 画点坐标
            color 点的颜色
返回值： 无
Function description: Draw a point at the specified position
Input data: x, y coordinates of the point
            color color of the point
Return value: None
*****/
void LCD_DrawPoint(uint16_t x,uint16_t y,uint16_t color)
{
    LCD_Address_Set(x,y,x,y);//设置光标位置 Set the cursor position
    LCD_WR_DATA(color);
}
...

```

bsp_spi.h

```

#ifndef _BSP_SPI_H__
#define _BSP_SPI_H__

#include "board.h"

//CS引脚的输出控制
//x=0时输出低电平
//x=1时输出高电平
//CS pin output control
//x=0 when output is low level
//x=1 when output is high level
#define SPI_CS(x) ( (x) ? DL_GPIO_setPins(CS1_PORT,CS1_PIN_17_PIN) :
DL_GPIO_clearPins(CS1_PORT,CS1_PIN_17_PIN) )

uint16_t w25q32_readID(void);//读取w25q32的ID Read the ID of w25q32

```

```

void w25q32_write(uint8_t* buffer, uint32_t addr, uint16_t numbyte);
//w25q32写数据 w25q32 write data
void w25q32_read(uint8_t* buffer, uint32_t read_addr, uint16_t
read_length); //w25q32读数据 w25q32 Read Data
#endif

```

SPI的初始化在SYSCONFIG中已经配置完成，但是我们还需要准备SPI读写步骤。为确保发送和接收数据成功，在发送时，需要确保发送缓冲区里的数据发送完毕，即发送缓冲区为空，才可以进行下一个数据的发送；在接收时，需要确保接收缓冲区里有数据才能够进行接收。

bsp_spi.c（这里只截取部分，详细内容请查看工程源码）

```

#include "bsp_spi.h"

uint8_t spi_read_write_byte(uint8_t dat)
{
    uint8_t data = 0;

    //发送数据 Sending Data
    DL_SPI_transmitData8(SPI_INST, dat);
    //等待SPI总线空闲 Wait for the SPI bus to be idle
    while(DL_SPI_isBusy(SPI_INST));
    //接收数据 Receiving Data
    data = DL_SPI_receiveData8(SPI_INST);
    //等待SPI总线空闲 Wait for the SPI bus to be idle
    while(DL_SPI_isBusy(SPI_INST));

    return data;
}

...

```

之后在empty.c文件中，写入以下代码

```

#include "ti_msp_dl_config.h"
#include "board.h"
#include "lcd_init.h"
#include "lcd.h"
#include "pic.h"
#include "bsp_spi.h"
#include <stdint.h>
#include <stdio.h>

int main(void)
{
    unsigned char buff[10] = {0};

    //开发板初始化 Development board initialization
    board_init();

    delay_ms(100); //等待部署 waiting for deployment

    //读取w25q32的ID Read the ID of w25q32
    printf("ID = %X\r\n", w25q32_readID());
}

```



```

//读取0地址的5个字节数据到buff Read 5 bytes of data from address 0 to buff
w25Q32_read(buff, 0, 7);

//串口输出读取的数据 Serial port outputs the read data
printf("buff = %s\r\n",buff);

//往0地址写入5个字节长度的数据 ABCD Write 5 bytes of data ABCD to address 0
w25Q32_write("Yahboom", 0, 7);
delay_ms(100);

//读取0地址的5个字节数据到buff Read 5 bytes of data from address 0 to buff
w25Q32_read(buff, 0, 7);

//串口输出读取的数据 Serial port outputs the read data
// printf("buff = %s\r\n",buff);

SYSCFG_DL_init();

LCD_Init();//LCD初始化 LCD Initialization
LCD_Fill(0,0,LCD_W,LCD_H,WHITE);
LCD_ShowPicture(20,45,120,29,gImage_pic1);
LCD_ShowString(10,0,"Hello!",BLACK,WHITE,16,0);
LCD_ShowChinese(50,20,"亚博智能",BLACK,WHITE,16,0);

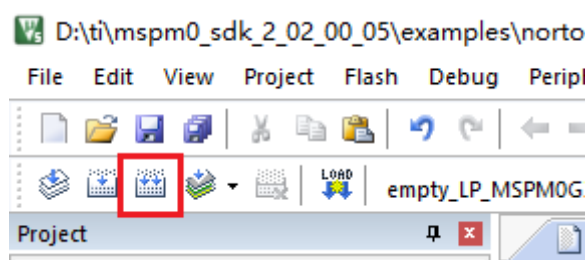
// 显示 buff 中的内容在同一行上 Display the contents of buff on the same line
int x = 54; // 起始 x 坐标 Starting x coordinate
for (int i = 0; i < 7; i++)
{
    char str[2] = {buff[i], '\0'}; // 每次取一个字符 Take one character at a
time
    LCD_ShowString(x, 36, str,BLACK, WHITE, 16, 0); // 显示字符 Display
Characters
    x += 8; // 每个字符的宽度为 8, x 坐标向右偏移 Each character is 8 wide and
has an x-coordinate offset to the right.
}

while (1)
{
}
}

```

5. 编译

点击 Rebuild 图标。出现以下提示表示编译完成，并且没有错误。



```

Generating Code (empty.syscfg)...
Unchanged D:\ti\mspm0_sdk_2_02_00_05\examples\nortos\LP_MSPM0G3507\driverlib\empty\ti_msp_dl_config.c...
Unchanged D:\ti\mspm0_sdk_2_02_00_05\examples\nortos\LP_MSPM0G3507\driverlib\empty\ti_msp_dl_config.h...
Unchanged D:\ti\mspm0_sdk_2_02_00_05\examples\nortos\LP_MSPM0G3507\driverlib\empty\Event.dot...
assembling startup_mspm0g350x_uvision.s...
compiling empty.c...
compiling ti_msp_dl_config.c...
linking...
Program Size: Code=544 RO-data=208 RW-data=0 ZI-data=352
FromELF: creating hex file...
".\Objects\empty_LP_MSPM0G3507_nortos_keil.axf" - 0 Error(s), 0 Warning(s).
Build Time Elapsed: 00:00:06

```

四、程序分析

- lcd.c

```

/*****
函数说明：在指定区域填充颜色
入口数据：xsta,ysta  起始坐标
           xend,yend  终止坐标
           color      要填充的颜色
返回值：  无
Function description: Fill the specified area with color
Input data: xsta, ysta starting coordinates
           xend, yend ending coordinates
           color the color to be filled
Return value: None
*****/
void LCD_Fill(uint16_t xsta,uint16_t ysta,uint16_t xend,uint16_t yend,uint16_t
color)
{
    uint16_t i,j;
    LCD_Address_Set(xsta,ysta,xend-1,yend-1); //设置显示范围 Set the display range
    for(i=ysta;i<yend;i++)
    {
        for(j=xsta;j<xend;j++)
        {
            LCD_WR_DATA(color);
        }
    }
}

```

`LCD_Fill` 函数在指定的起始坐标 (`xsta, ysta`) 和终止坐标 (`xend, yend`) 区域内填充指定颜色，通过设置显示范围并循环写入颜色数据完成区域填充操作。

```

/*****
函数说明：显示图片
入口数据：x,y 起点坐标
           length 图片长度
           width  图片宽度
           pic[]  图片数组
返回值：  无
Function description: Display image
Input data: x, y starting point coordinates
           length Image length
           width Image width
           pic[] Image array
Return value: None
*****/

```

```

void LCD_ShowPicture(uint16_t x,uint16_t y,uint16_t length,uint16_t width,const
uint8_t pic[])
{
    uint16_t i,j;
    uint32_t k=0;
    LCD_Address_Set(x,y,x+length-1,y+width-1);
    for(i=0;i<length;i++)
    {
        for(j=0;j<width;j++)
        {
            LCD_WR_DATA8(pic[k*2]);
            LCD_WR_DATA8(pic[k*2+1]);
            k++;
        }
    }
}

```

LCD_ShowPicture 函数在指定起点 (x, y) 绘制图片，根据给定的长度和宽度设置显示区域，并通过遍历图片数组，将每个像素点的颜色数据写入 LCD，从而完成图片显示。

```

/*****
    函数说明：显示字符串
    入口数据：x,y显示坐标
                *p 要显示的字符串
                fc 字的颜色
                bc 字的背景色
                sizey 字号
                mode: 0非叠加模式 1叠加模式
    返回值： 无
    Function description: Display string
    Input data: x, y display coordinates
                *p string to be displayed
                fc color of the word
                bc background color of the word
                sizey font size
                mode: 0 non-overlay mode 1 overlay mode
    Return value: None
*****/
void LCD_ShowString(uint16_t x,uint16_t y,const uint8_t *p,uint16_t fc,uint16_t
bc,uint8_t sizey,uint8_t mode)
{
    while(*p!='\0')
    {
        LCD_ShowChar(x,y,*p,fc,bc,sizey,mode);
        x+=sizey/2;
        p++;
    }
}

```

LCD_ShowString 函数用于在指定坐标 (x, y) 显示字符串，依次调用 **LCD_ShowChar** 函数绘制字符串中的每个字符，同时根据字号调整字符间距，支持叠加模式和非叠加模式显示。

```

/*****
    函数说明：显示汉字串
    入口数据：x,y显示坐标
                *s 要显示的汉字串

```

```

        fc 字的颜色
        bc 字的背景色
        sizey 字号 可选 16 24 32
        mode: 0非叠加模式 1叠加模式

返回值: 无
Function description: Display Chinese character string
Input data: x, y display coordinates
            *s Chinese character string to be displayed
            fc character color
            bc character background color
            sizey font size optional 16 24 32
            mode: 0 non-overlay mode 1 overlay mode

Return value: None
*****/
void LCD_ShowChinese(uint16_t x,uint16_t y,uint8_t *s,uint16_t fc,uint16_t
bc,uint8_t sizey,uint8_t mode)
{
    while(*s!=0)
    {
        LCD_ShowChinese16x16(x,y,s,fc,bc,sizey,mode);
        s+=2;
        x+=sizey;
    }
}

```

`LCD_ShowChinese` 函数用于在指定坐标 `(x, y)` 显示汉字串，通过逐个调用

`LCD_ShowChinese16x16` 函数绘制每个汉字，并根据字号调整下一个汉字的显示位置，支持叠加模式和非叠加模式显示。

- bsp_spi.c

```

/*****
* 函数名称: w25q32_read
* 函数功能: 读取W25Q32的数据
* 传入参数: buffer=读出数据的保存地址  read_addr=读取地址  read_length=读去长度
* 函数返回: 无
* 作者: LC
* 备注: 无
* Function name: w25q32_read
* Function function: Read w25q32 data
* Input parameters: buffer = storage address of read data read_addr = read
address read_length = read length
* Function return: None
* Author: LC
* Notes: None
*****/
void w25q32_read(uint8_t* buffer,uint32_t read_addr,uint16_t read_length)
{
    uint16_t i;
    //拉低CS端为低电平 Pull the CS end to a low level
    SPI_CS(0);
    //发送指令03h Send instruction 03h
    spi_read_write_byte(0x03);
    //发送24位读取数据地址的高8位
    // Send the high 8 bits of the 24-bit read data address
    spi_read_write_byte((uint8_t)((read_addr)>>16));
    //发送24位读取数据地址的中8位

```

```

        // Send the middle 8 bits of the 24-bit read data address
        spi_read_write_byte((uint8_t)((read_addr)>>8));
        //发送24位读取数据地址的低8位
        // Send the low 8 bits of the 24-bit read data address
        spi_read_write_byte((uint8_t)read_addr);
        //根据读取长度读取出地址保存到buffer中
        // Read the address according to the read length and save it in the
buffer
        for(i=0;i<read_length;i++)
        {
            buffer[i]= spi_read_write_byte(0xFF);
        }
        //恢复CS端为高电平 Restore the CS end to a high level
        SPI_CS(1);
    }

```

w25q32_read 函数用于从 W25Q32 闪存芯片中读取数据。该函数首先通过拉低 CS 引脚来启动 SPI 通信，然后发送读取命令（0x03）以及读取地址的高、中、低三字节信息。接着，根据指定的读取长度，逐字节地读取数据并存储到提供的缓冲区（**buffer**）中。最后，函数将 CS 引脚恢复为高电平，结束数据传输。这个过程确保了从指定地址读取指定长度的数据。

```

/*****
* 函 数 名 称: w25q32_write
* 函 数 功 能: 写数据到w25q32进行保存
* 传 入 参 数: buffer=写入的数据内容          addr=写入地址          numbyte=写入数据的长度
* 函 数 返 回: 无
* 作 者: LC
* 备 注: 无
* Function name: w25q32_write
* Function function: write data to w25q32 for storage
* Input parameters: buffer = data content to be written addr = write address
numbyte = length of written data
* Function return: None
* Author: LC
* Notes: None
*****/
void w25q32_write(uint8_t* buffer, uint32_t addr, uint16_t numbyte)
{
    unsigned int i = 0;
    //擦除扇区数据 Erase sector data
    w25q32_erase_sector(addr/4096);
    //写使能 Write enable
    w25q32_write_enable();
    //忙检测 Busy detection
    w25q32_wait_busy();
    //写入数据 write data
    //拉低CS端为低电平 Pull CS to low level
    SPI_CS(0);
    //发送指令02h Send instruction 02h
    spi_read_write_byte(0x02);
    //发送写入的24位地址中的高8位
    // Send the high 8 bits of the 24-bit address to be written
    spi_read_write_byte((uint8_t)((addr)>>16));
    //发送写入的24位地址中的中8位
    // Send the middle 8 bits of the 24-bit address to be written
    spi_read_write_byte((uint8_t)((addr)>>8));
    //发送写入的24位地址中的低8位

```

```

// Send the low 8 bits of the 24-bit address to be written
spi_read_write_byte((uint8_t)addr);
//根据写入的字节长度连续写入数据buffer
// Continuously write data buffer according to the length of the written
byte
for(i=0;i<numbyte;i++)
{
    spi_read_write_byte(buffer[i]);
}
//恢复CS端为高电平 Restore CS end to high level
SPI_CS(0);
//忙检测 Busy detection
w25q32_wait_busy();
}

```

w25q32_write 函数用于将数据写入 W25Q32 闪存芯片。首先，函数会擦除目标地址所在的扇区数据。然后，启用写操作并检查芯片是否忙碌。接下来，函数通过 SPI 接口向芯片发送写入命令（0x02）和数据地址（24 位地址），并根据指定的写入长度逐字节地写入数据。最后，函数等待写操作完成并恢复 CS 引脚为高电平，确保数据正确写入并且芯片处于空闲状态。

- empty.c

```

int main(void)
{
    unsigned char buff[10] = {0};

    //开发板初始化 Development board initialization
    board_init();

    delay_ms(100); //等待部署 waiting for deployment

    //读取W25Q32的ID Read the ID of w25q32
    printf("ID = %x\r\n", w25q32_readID());

    //读取0地址的5个字节数据到buff Read 5 bytes of data from address 0 to buff
    w25q32_read(buff, 0, 7);

    //串口输出读取的数据 Serial port outputs the read data
    printf("buff = %s\r\n", buff);

    //往0地址写入5个字节长度的数据 ABCD Write 5 bytes of data ABCD to address 0
    w25q32_write("Yahboom", 0, 7);
    delay_ms(100);

    //读取0地址的5个字节数据到buff Read 5 bytes of data from address 0 to buff
    w25q32_read(buff, 0, 7);

    //串口输出读取的数据 Serial port outputs the read data
    // printf("buff = %s\r\n", buff);

    SYSCFG_DL_init();

    LCD_Init(); //LCD初始化 LCD Initialization
    LCD_Fill(0, 0, LCD_W, LCD_H, WHITE);
    LCD_ShowPicture(20, 45, 120, 29, gImage_pic1);
    LCD_ShowString(10, 0, "Hello!", BLACK, WHITE, 16, 0);
    LCD_ShowChinese(50, 20, "亚博智能", BLACK, WHITE, 16, 0);
}

```

```

// 显示 buff 中的内容在同一行上 Display the contents of buff on the same line
int x = 54; // 起始 x 坐标 Starting x coordinate
for (int i = 0; i < 7; i++)
{
    char str[2] = {buff[i], '\0'}; // 每次取一个字符 Take one character at a
time
    LCD_ShowString(x, 36, str, BLACK, WHITE, 16, 0); // 显示字符 Display
Characters
    x += 8; // 每个字符的宽度为 8, x 坐标向右偏移 Each character is 8 wide and
has an x-coordinate offset to the right.
}

while (1)
{

}
}

```

该程序的主要功能是初始化开发板，读取和写入 W25Q32 闪存芯片的数据，并通过 LCD 显示读取的内容。首先，程序初始化开发板和相关硬件。然后，它读取 W25Q32 的芯片 ID 并通过串口输出。接下来，程序从闪存的起始地址（0 地址）读取 7 个字节数据到 `buff` 数组，并在串口输出这些数据。接着，它将字符串 "Yahboom" 写入闪存的 0 地址，并再次读取并显示该数据。最后，程序初始化 LCD 屏幕，显示欢迎信息和图片，并将读取的 `buff` 数据按字符显示在屏幕上的同一行。

五、实验现象

程序下载完成之后，在 LCD 显示屏上会分行显示文字“Hello!”、“亚博智能”、flash 内的数据及亚博智能的 logo。



